🏦 CrestBank AI Personal Finance Advisor

Multi-Agent Architecture, LangGraph Orchestration, Tool Calling, Memory & Guardrails — Technical Submission Report

LangGraph StateGraph | 4 Specialized AI Agents | Google Gemini + Deterministic Fallback | Pydantic V2 State Contracts | Dual-Tier Memory | Dual-Mode Guardrails | Streamlit Conversational UI

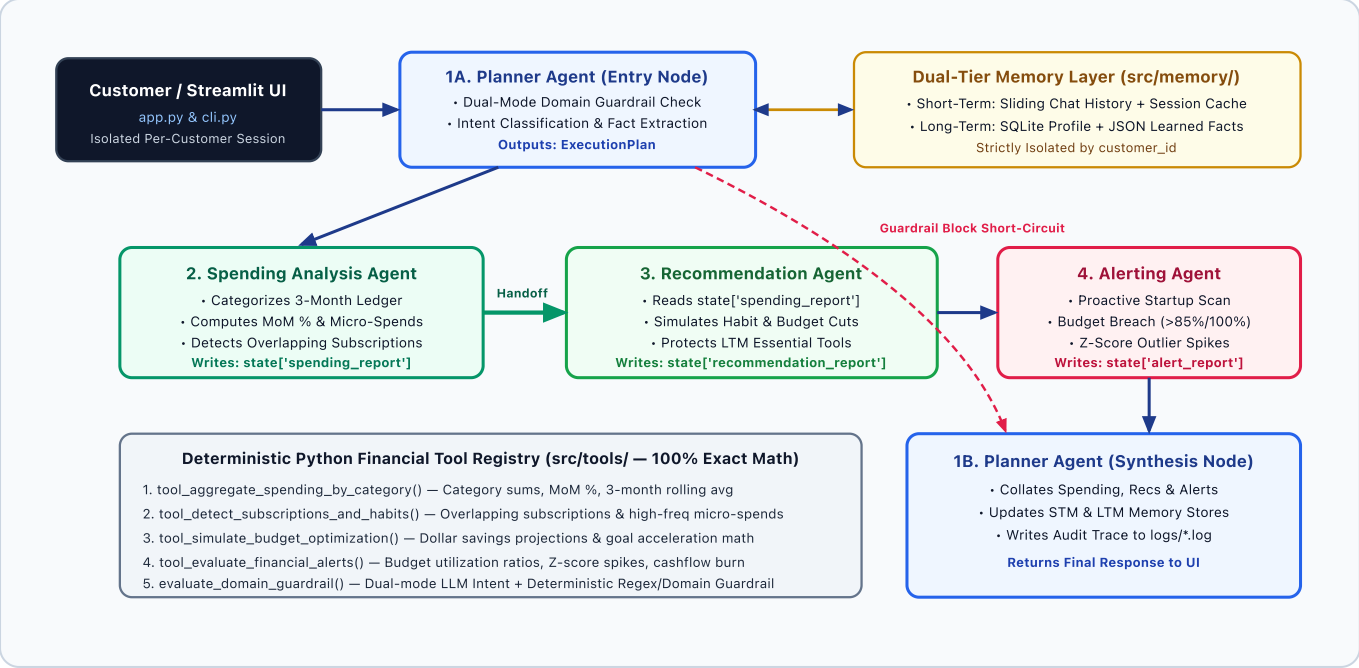
1. Executive Summary & Problem Statement

Traditional retail banking chatbots are limited to **static monthly account statements and passive one-way transaction notifications**. Customers see *what* they spent after the fact, but receive no autonomous reasoning across historical patterns, no root-cause categorization of spending surges, and no quantified guidance tied to their life goals.

The **CrestBank AI Personal Finance Advisor** upgrades this legacy experience into an autonomous, multi-agent financial advisory system built on **LangGraph** and **Google Gemini**. It coordinates four specialized agents that read multi-month transaction histories, detect behavioral anomalies and subscription creep, enforce strict inter-agent data dependencies, persist customer preferences in Dual-Tier Memory, and block off-topic requests via a Dual-Mode Domain Guardrail.

> **Mandatory Architectural Dependency Satisfied:** The system enforces at both the `LangGraph StateGraph` edge level and runtime validation level that the **Recommendation Agent** strictly receives its analytical input (`SpendingAnalysisReport`) directly from the **Spending Analysis Agent**.

2. System Architecture & LangGraph Orchestration Diagram



3. Detailed Specification of the 4 Specialized Agents

1. Planner Agent `src/agents/planner_agent.py`

Role: Front-door supervisor, guardrail gatekeeper, execution planner, and final response synthesizer.

- **Entry Node (`planner_plan_node`):** Runs the Dual-Mode Guardrail, extracts new user facts/budget updates into Long-Term Memory, classifies intent, and outputs a typed `ExecutionPlan` .
- **Dependency Enforcement:** Whenever recommendations are needed, forces `run_spending_agent = True` first.
- **Synthesis Node (`planner_synthesize_node`):** Collates outputs from all executed sub-agents into markdown tables and bulleted action plans.

2. Spending Analysis Agent `src/agents/spending_agent.py`

Role: Transaction data scientist & behavioral pattern recognizer.

- **Tools Called:** `tool_aggregate_spending_by_category` and `tool_detect_subscriptions_and_habits` .
- **Capabilities:** Categorizes 3 months of transactions across 8 categories, calculates MoM dollar/percentage shifts, detects overlapping subscriptions (e.g., 3 video streaming services), and quantifies high-frequency micro-spend habits (e.g., 11x monthly food delivery orders).
- **Output Contract:** `SpendingAnalysisReport` written to `state["spending_report"]` .

3. Recommendation Agent `src/agents/recommendation_agent.py`

Role: Personalized financial strategist & budget optimizer.

- **Strict Input Contract:** Consumes `SpendingAnalysisReport` directly from the Spending Analysis Agent (raises `ValueError` if missing).
- **Tools Called:** `tool_simulate_budget_optimization` .
- **Capabilities:** Computes monthly/annual savings from right-sizing micro-spend habits, consolidating redundant subscriptions (while respecting LTM constraints like keeping Adobe/Figma), normalizing category spikes, and sweeping idle checking cash into a 4.50% APY vault.

4. Alerting Agent `src/agents/alerting_agent.py`

Role: Proactive & reactive overspending and risk monitor.

- **Dual Modes:** Runs *proactively* when a customer opens the chatbot (upgrading passive bank notifications) and *reactively* as a node inside the LangGraph workflow.
- **Tools Called:** `tool_evaluate_financial_alerts` .
- **Signals Flagged:** Category budget breaches (≥100% Critical, ≥85% Warning), statistical single-transaction outliers (Z-score ≥ 2.2), subscription creep, and low checking liquidity burn risks.

4. Tool Registry, Dual-Tier Memory & Guardrail Engineering

Tool / Module	Invoked By	Algorithmic & Functional Description
`tool_aggregate_spending_by_category` (`transaction_tools.py`)	Spending Analysis Agent	Queries SQLite transactions across `2026-08`, `2026-09`, and `2026-10`; computes category totals, 3-month rolling averages, budget utilization %, Month-over-Month (MoM) deltas, and top merchants per category.
`tool_detect_subscriptions_and_habits` (`transaction_tools.py`)	Spending Analysis Agent	Identifies recurring charges, flags overlapping services in the same subcategory (e.g., multiple video/music apps), and clusters high-frequency micro-spends (≥3 txns/mo in Food Delivery, Coffee Shops, or Rideshare).
`tool_simulate_budget_optimization` (`simulation_tools.py`)	Recommendation Agent	Takes `SpendingAnalysisReport` + customer goals; simulates 45%–50% habit frequency reductions, redundant subscription removals (filtering out LTM-protected work tools), and APY cash sweeps; computes new savings rate % and months saved toward primary goal.
`tool_evaluate_financial_alerts` (`anomaly_tools.py`)	Alerting Agent	Evaluates 4 risk dimensions: (1) Budget utilization ≥85% / ≥100%, (2) Single-transaction statistical outliers ($Z = (x - \mu)/\sigma \ge 2.2$), (3) Subscription creep, and (4) Low checking buffer / idle cash opportunity.
`evaluate_domain_guardrail` (`guardrail_tools.py`)	Planner Agent (Entry)	**Dual-Mode Guardrail:** Uses Gemini semantic intent classification when `GOOGLE_API_KEY` is active, or deterministic regex blocklists (recipes/pasta, medical/acne, coding, trivia, prompt injection) + positive financial keyword verification when offline.
`MemoryManager` (`memory_manager.py`)	Planner & Recommendation Agents	**Short-Term Memory:** Per-customer sliding window (12 turns) + cached session reports. **Long-Term Memory:** Persistent SQLite profile + `long_term_memory.json` storing learned facts and dynamic budget updates (e.g., *"set my dining budget to $500"*).

5. Dual-Mode Domain Guardrails & Safety Architecture (`src/tools/guardrail_tools.py`)

Because a banking advisor must never hallucinate medical, culinary, or off-topic advice—or waste compute running financial database tools on irrelevant prompts—the system enforces a **Dual-Mode Domain Guardrail** inside the **Planner Agent's Entry Node (`planner_plan_node`)** before any sub-agent is invoked:

Mode 1: Live LLM Semantic Intent Guardrail

When `GOOGLE_API_KEY` Is Set

Mechanism: Invokes Google Gemini (`temperature=0.0`) as a zero-shot semantic compliance classifier before planning.

- **Classification Contract:** Returns strictly `ALLOW | <reason>` or `BLOCK | <off-topic category & reason>`.
- **Scope Enforced:** Permits personal finance, spending analysis, budgeting, savings goals, subscriptions, and account inquiries; blocks all non-financial intents and adversarial prompt injections.

Mode 2: Deterministic Pattern & Scope Guardrail

Offline / No API Key

Mechanism: Two-stage deterministic rule engine ensuring 100% protection even without an external LLM key:

- **Stage A — Categorized Off-Topic Regex Blocklist:** Blocks *Cooking & Recipes* (e.g., `"give me a pasta receipe"`), *Medical & Health* (e.g., `"how to treat my acne"`), *Coding/Homework*, *General Trivia/Weather*, and *Prompt Injection / Jailbreaks*.
- **Stage B — Positive Financial Keyword Verification:** Rejects queries lacking any banking, spending, budget, or transaction terminology.

> **LangGraph Short-Circuit Routing on Guardrail Block:** When `evaluate_domain_guardrail()` returns `is_allowed = False`, the Planner sets `user_intent = "BLOCKED_OFF_TOPIC"` and `route_after_planner()` routes directly from `planner_plan_node → planner_synthesize_node`. **Zero sub-agents (Spending Analysis, Recommendation, Alerting) or SQLite transaction tools are executed**, and the blocked event is logged as a `[GUARDRAIL]` step in `logs/`.

6. Multi-Persona Mock Dataset & Planted Financial Patterns

To rigorously demonstrate autonomous reasoning across diverse financial situations, `src/data/mock_generator.py` seeds SQLite (`data_store/bank_advisor.db`) with **3 customer personas across 3 months (~155 realistic transactions)**:

Persona	Financial Profile & Goals	Planted Transaction Patterns Autonomous Agents Discover
Alex Rivera (`CUST-001`) *Young Tech Pro*	Income: `$6,200/mo` Checking: `$3,420` Goal: House Down Payment (`$6.8k / $20k`)	• **Food Delivery Surge:** Food & Dining jumps `+72.6% MoM` (`$430` → `$742` , `165%` of `$450` budget) driven by 11 DoorDash/UberEats orders. • **Subscription Creep:** 6 recurring charges (`$136.94/mo` vs `$80` budget) with overlapping video (StreamFlix, HuluPlus, MaxCinema) and music apps. • **Recommendation Output:** Unlocks **`$430.71/mo ($5,168.52/yr)`** , boosting savings rate from `28.2%` → `35.2%` .
Priya Sharma (`CUST-002`) *Family Manager*	Income: `$8,500/mo` Checking: `$14,200` Goal: 6-Mo Emergency Fund (`$9.5k / $25k`)	• **Statistical Z-Score Outlier:** One-off `$680.00` emergency dishwasher purchase at HomeDepot (`2.5x` category mean, $Z = 2.2$). • **Utility Bill Surge:** Electricity & gas surged `+60.3% MoM` to `$388` (`149%` of budget). • **Idle Cash Sweep:** Holds `$14,200` in checking at 0.01% APY; recommends sweeping `$7,400` into 4.50% APY Vault (`+$333/yr` risk-free interest).
Marcus Vance (`CUST-003`) *Freelance Designer*	Income: Variable (`$3,600` in Oct) Checking: `$890` Goal: Pay Off High-APR Card (`$2,400`)	• **Critical Cash-Flow Burn Alert:** October freelance income dip + low checking buffer (`$890`) triggers a `CRITICAL` liquidity warning. • **Dual Micro-Spend Drain:** 10x specialty coffee visits (`$115/mo`) + 8x Uber/Lyft rides (`$276/mo`) = `$391/mo ($4,692/yr)` drain. • **LTM-Protected Work Tools:** Long-Term Memory notes `Adobe CC` & `Figma` are essential work tools, so Recommendation Agent protects them from cancellation.

7. Key Architectural Decisions & Engineering Trade-Offs

Architectural Decision	Why Chosen (Pros)	Trade-Offs Mitigated
ADR-01: LangGraph `StateGraph` vs. Peer-to-Peer Swarm	Guarantees deterministic execution order and enforces via conditional edges that `Recommendation Agent` always receives `state["spending_report"]` from `Spending Analysis Agent`.	Requires explicit state schema definition upfront, trading free-form agent chatter for 100% banking auditability and zero infinite loops.
ADR-02: Deterministic Python Tool Math vs. LLM Token Math	Eliminates LLM arithmetic hallucinations. Every dollar sum, MoM %, Z-score, and compound savings projection is 100% exact and executes in <2ms.	Requires dedicated Python tool functions in `src/tools/` rather than dumping raw transaction CSVs into prompts.
ADR-03: Typed Pydantic V2 Inter-Agent Contracts	Prevents "telephone-game" context drift between agents; validates field types at runtime.	Schema updates require modifying `src/schemas/models.py`.
ADR-04: Dual-Mode Gemini Engine + Offline Fallback	Supports live Google Gemini when `GOOGLE_API_KEY` is set, while guaranteeing 100% offline execution for evaluators without an API key.	Maintains both LLM prompts and structured synthesis templates.
ADR-05: Background File Trace Logging (`logs/`)	Keeps the customer-facing Streamlit UI clean while recording full step-by-step agent traces to `logs/agent_traces.jsonl` and `logs/<CUST_ID>_traces.log`.	Developers inspect trace logs in `logs/` or via `python cli.py --demo` rather than cluttering the end-user chat bubble.
ADR-06: Dual-Mode Domain & Safety Guardrail	Prevents off-topic queries (recipes, medical/acne advice, coding, trivia) and prompt injections using Gemini intent classification when online and deterministic regex + domain-keyword rules when offline.	Short-circuits LangGraph directly to refusal synthesis, saving compute and protecting bank compliance boundaries.